

JAVA PROGRAMMING
Practical Examination Questions

40 Questions - Units 1, 2, 3

Duration: 1 Hour | Maximum Marks: 50 per Question

Unit 1 (OOP Fundamentals), Unit 2 (Inheritance & Interfaces), Unit 3 (Exception Handling & I/O)

UNIT 1: OOP FUNDAMENTALS

QUESTION 1: Employee Management System using Encapsulation

Design a Java program to create an Employee class with private variables (empId, name, salary) and public getter/setter methods. Implement data validation in setters to ensure salary is positive. Create a main method to demonstrate encapsulation.

QUESTION 2: Student Class with Constructor and Methods

Create a Java class Student with student ID, name, and marks in three subjects. Use a constructor and methods to calculate total and average and display the student details.

QUESTION 3: BankAccount Encapsulation

Create a Java class BankAccount using encapsulation. Include private account details and public methods for deposit, withdrawal, and balance enquiry. Demonstrate the class using objects.

QUESTION 4: Static Members and Object Counter

Write a Java program to demonstrate static members. Create a Student class with a static counter that counts the number of Student objects created and display the count.

QUESTION 5: Constructor Overloading and this keyword

Create a Book class with multiple constructors (default, one parameter, two parameters). Use 'this' keyword to call constructors and initialize objects. Demonstrate constructor chaining with different parameter combinations.

QUESTION 6: Singleton Pattern with Static Members

Implement a Database class as a singleton using static members. Ensure only one instance can be created. Demonstrate access to static methods and the single instance across multiple references in the main method.

QUESTION 7: Inner Classes and ArrayList

Create an outer class University with an inner class Department. Use ArrayList to store multiple Department objects. Implement methods to add, remove, and display departments. Demonstrate object relationships.

QUESTION 8: Lambda Expressions and Functional Interfaces

Create a functional interface `Calculator` with a `calculate(int a, int b)` method. Implement addition, subtraction, multiplication using lambda expressions. Demonstrate passing lambda expressions as parameters to a processing method.

QUESTION 9: Streams API: Filtering and Mapping

Create a list of `Student` objects with attributes (`id`, `name`, `marks`). Use Streams API to filter students with `marks > 75`, map to uppercase names, and collect results. Display filtered and mapped results.

QUESTION 10: Functional Interface with Lambda Expression

Write a Java program using a functional interface and a lambda expression to calculate the square of a number and the sum of two numbers.

QUESTION 11: Streams API: Filter, Map, Reduce Operations

Write a Java program using the Streams API to read a list of integers, filter the even numbers, square them, and display the resulting values.

QUESTION 12: Object Cloning: Deep Copy Implementation

Create a `Person` class with `Address` as a nested object. Implement `Cloneable` interface and override `clone()` method for deep copying. Verify that modifying cloned object doesn't affect original. Handle `CloneNotSupportedException`.

QUESTION 13: String Manipulation and StringBuilder

Compare String concatenation using `+` operator vs `StringBuffer` vs `StringBuilder` for performance. Create a program that builds a large string using all three methods and measure their performance. Display comparison results.

UNIT 2: INHERITANCE AND INTERFACES

QUESTION 14: Single Inheritance: Vehicle Hierarchy

Create a `Vehicle` superclass with properties (`vehicleId`, `brand`) and methods (`start()`, `stop()`). Extend it with a `Car` subclass that overrides the `start()` method with car-specific implementation. Demonstrate polymorphism with a main method.

QUESTION 15: Multi-level Inheritance: Animal Classification

Implement multi-level inheritance with `Animal` (superclass) → `Mammal` (intermediate class) → `Dog` (subclass). Each class should override a `sound()` method. Use the protected access specifier appropriately. Test with object creation and method calls.

QUESTION 16: Abstract Classes and Methods: Shape Calculator

Create an abstract `Shape` class with abstract method `calculateArea()`. Implement concrete classes `Circle` and `Rectangle`. Use abstract class features to ensure all shapes must implement `calculateArea()`. Demonstrate with multiple shape objects.

QUESTION 17: Interface Implementation: Sortable Objects

Design an interface Sortable with method compare(). Implement it in Student and Product classes. Demonstrate how multiple classes can implement the same interface with different logic. Create arrays and sort them using the interface.

QUESTION 18: Figure Superclass with Polymorphism

Create a superclass Figure with dimensions and an area() method. Derive Rectangle and Triangle classes and override area() to calculate their respective areas. Demonstrate runtime polymorphism.

QUESTION 19: Employee Superclass - Salary Calculation

Create an Employee superclass with employee ID, name, and basic pay. Derive Programmer and Professor classes and calculate DA, HRA, PF, gross salary, and net salary for each employee.

QUESTION 20: Multi-level Employee Inheritance - Pay Slip

Create an Employee superclass with employee details and derive Programmer, AssistantProfessor, AssociateProfessor, and Professor classes. Generate a pay slip for a selected employee designation.

QUESTION 21: Abstract Shape Class with printArea()

Create an abstract class Shape with an abstract method printArea(). Derive Rectangle, Triangle, and Circle classes and implement printArea() in each class to display the corresponding area.

QUESTION 22: Abstract Vehicle Class with start()

Create an abstract class Vehicle with an abstract method start(). Derive Car and Bike classes and demonstrate abstraction and method overriding.

QUESTION 23: Interface - Payment (Multiple Implementation)

Define an interface Payment with a method pay(). Implement the interface using CreditCardPayment and UPIPayment classes and demonstrate polymorphism.

QUESTION 24: Multiple Interface Implementation

Define two interfaces Printable and Showable. Create a class Report that implements both interfaces and demonstrate multiple interface implementation.

QUESTION 25: Interface Shape with area() method

Create an interface Shape with an area() method. Implement it in Rectangle and Circle classes and display the areas using interface references.

QUESTION 26: Protected Members - Inheritance Demo

Write a Java program to demonstrate inheritance and the use of protected members. Create a superclass Person and a subclass Student that accesses the protected member.

QUESTION 27: Final Keyword - Variables, Methods, Classes

Write a Java program to demonstrate the use of final variables, final methods, and a final class. Include suitable examples and display the results.

QUESTION 28: ArrayList Operations

Write a Java program using an ArrayList to store student names. Add, remove, search, and display names from the list.

QUESTION 29: String Operations - Manipulation and Methods

Write a Java program to demonstrate String operations. Read a string and perform length, character access, substring, comparison, replacement, and reverse operations.

UNIT 3: EXCEPTION HANDLING AND I/O

QUESTION 30: Exception Handling - Division by Zero (try-catch-finally)

Write a Java program to handle division by zero using try, catch, and finally blocks. Display suitable messages for normal execution and exception handling.

QUESTION 31: Custom Exception - InvalidAgeException

Create a user-defined exception InvalidAgeException. Read an age and throw the exception if the age is negative. Handle the exception using try-catch and display an appropriate message.

QUESTION 32: Custom Exception - InvalidMarkException

Write a Java program to create a custom exception InvalidMarkException. Read a mark and throw the exception if the mark is outside the range 0 to 100.

QUESTION 33: BufferedReader - Read Text File Line by Line

Write a Java program to read a text file using BufferedReader and display its contents line by line. Also count and display the number of lines in the file.

QUESTION 34: File Write and Read - Character Stream I/O

Write a Java program to write employee details to a text file and then read the file using a character stream. Display the stored details on the console.

QUESTION 35: Multiple Exception Handling

Write a Java program to handle multiple exceptions using try-catch-finally blocks. Create a method that performs array access, string parsing, and type casting. Implement multiple catch blocks for ArrayIndexOutOfBoundsException, NumberFormatException, and ClassCastException.

QUESTION 36: Exception Hierarchy and Multiple Catch Blocks

Create a method that performs array access, string parsing, and type casting. Implement multiple catch blocks for different exceptions (ArrayIndexOutOfBoundsException, NumberFormatException, ClassCastException).

QUESTION 37: Finally Block and Resource Management

Implement exception handling with try-catch-finally where finally block always executes. Demonstrate with file operations and show that finally runs even when exception occurs.

QUESTION 38: File Operations - Copy File Using Streams

Write a Java program to copy one file to another using `FileInputStream` and `FileOutputStream`. Include exception handling and display confirmation of file copy operation.

QUESTION 39: Byte Stream Operations - Serialization

Create a class `Employee` that implements `Serializable`. Write employee objects to a file using `ObjectOutputStream`. Read objects back using `ObjectInputStream`. Display deserialized object properties.

QUESTION 40: Exception Re-throwing and Chaining

Create a parent exception `BankException` and child exceptions `InsufficientFundsException` and `InvalidAccountException`. Implement in a `Bank` class with `deposit/withdraw` methods. Handle exceptions in `main` method.